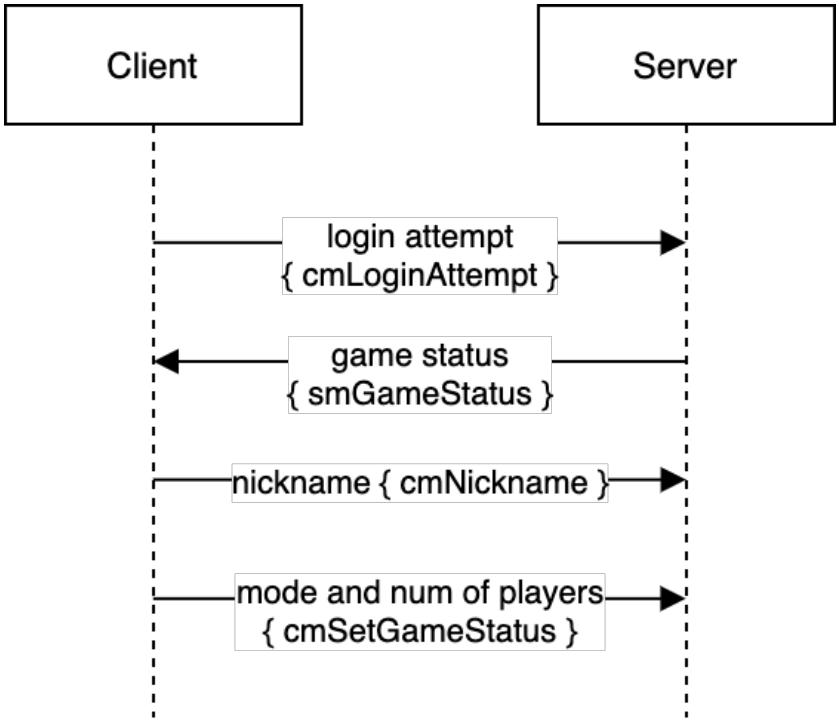
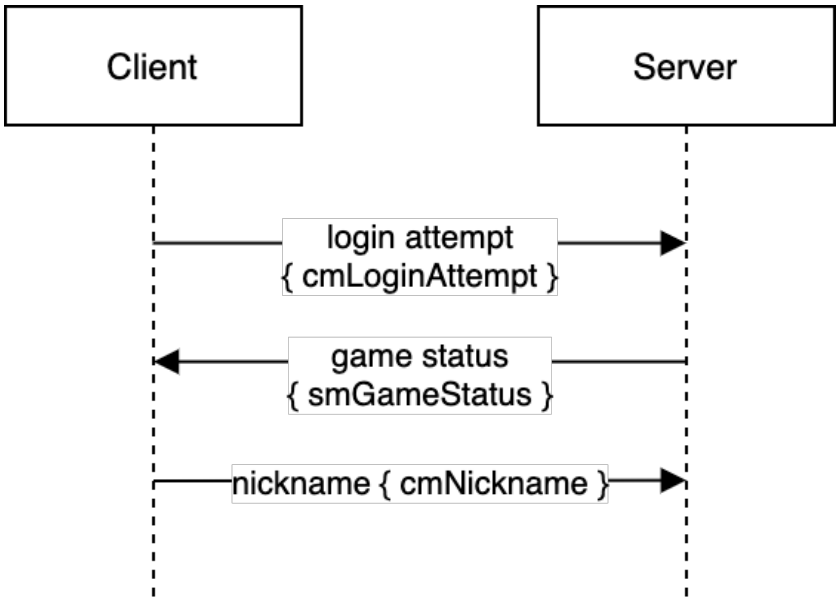
Set game

The server will set the game up with the client that made the first connection. It will ask to the first player his nickname, the mode, and the number of players.



Join game

Every client that wants to connect to the created match, will try to connect to the server with the message “login attempt”. Then the server asks to set the nickname sending him the nicknames already taken, the number of players needed for that match and the match mode.

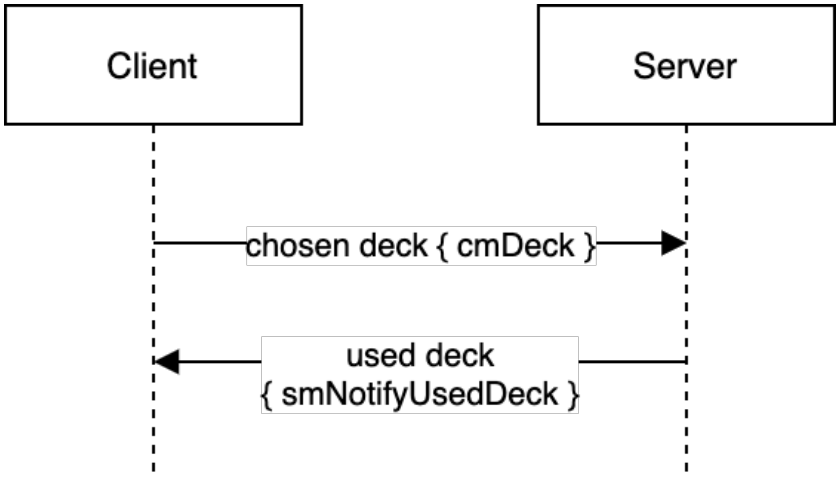


Waiting players

The players in the game wait until the total number of players is reached. Every time a new player joins the game, the server notifies the client with a message.

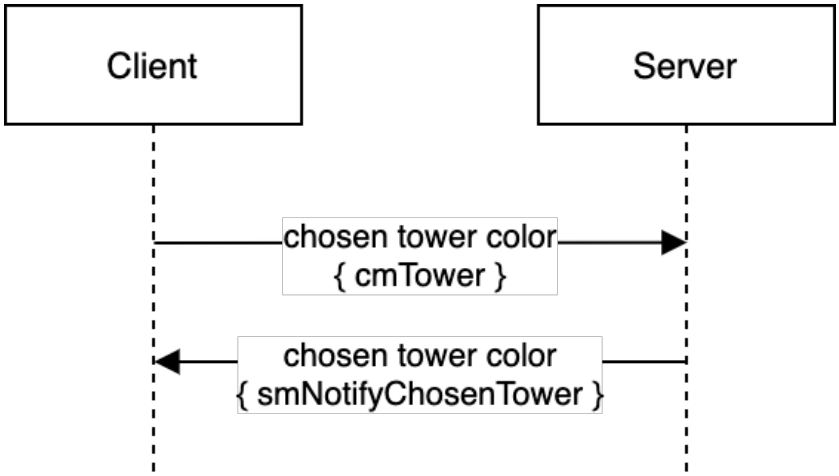
Choose deck

After the last client has logged in and has set the nickname, the client selects a deck.
The serve, then, notifies the other clients of the chosen deck.



Choose tower

The client chooses the tower's color. The server notifies the other players of the chosen color and also says who chose it.
Then the client that knows that it's his turn (the next one in order of login) restarts from the "choose deck" point.

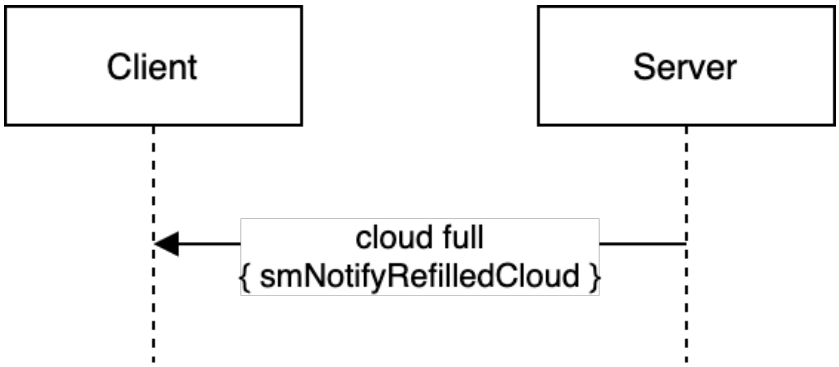


Planning phase

- fill cloud;
- choose support card.

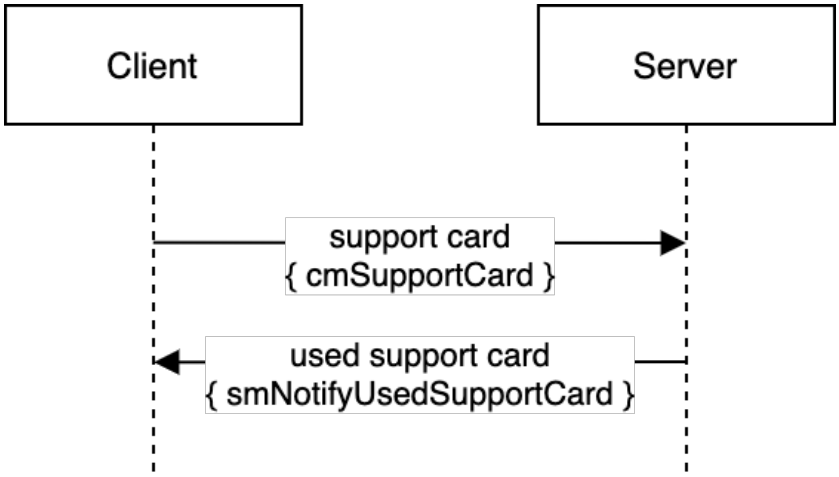
Fill cloud

The server draws three students from the bag and puts them on each cloud and notifies all the clients.



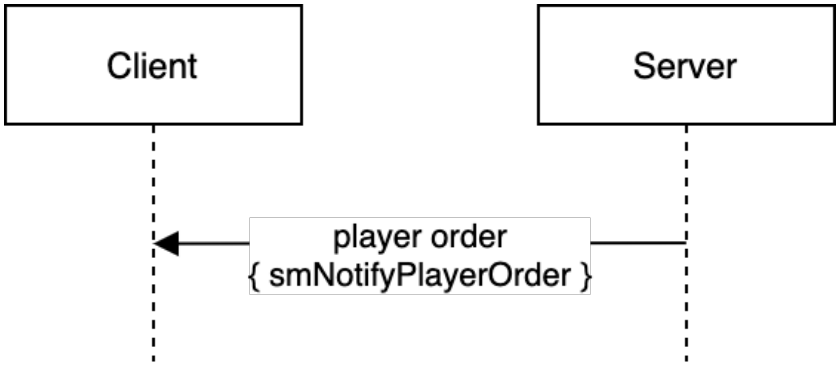
Choose support card

Successively the server asks to each client to select the support card.
Then the server notifies to the other clients connected to the game the support card used.



Player order

After all the clients have drawn a support card, the server will notify the clients with the players order (chosen in base of the support card value).

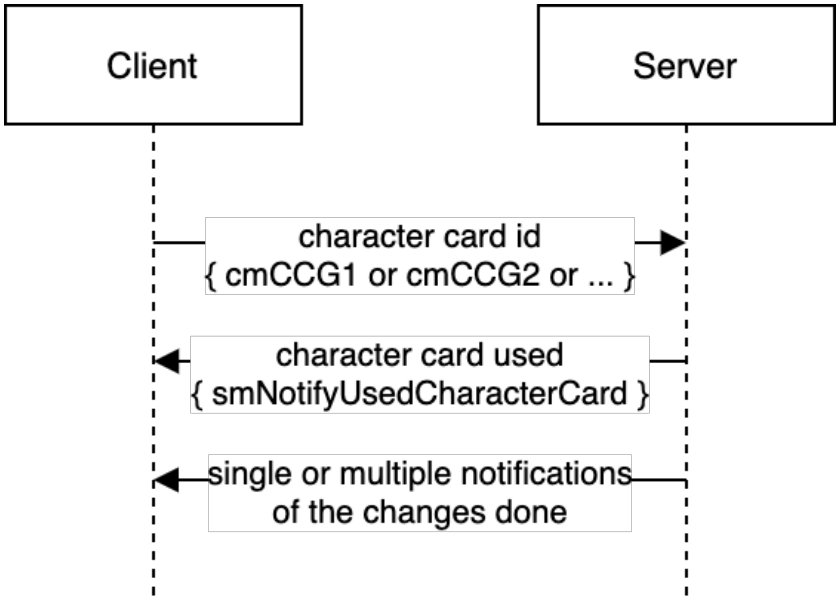


Action phase

- activate character card;
- students movements;
- move mother nature;
- choose cloud.

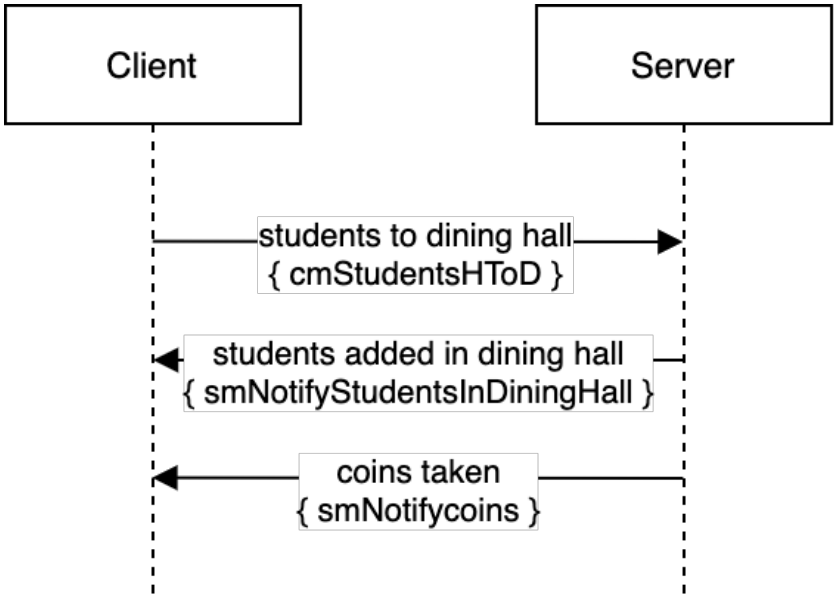
Activate character card

(Only in expert mode)
If the current player wants to activate a character card, he chooses one of the available ones and sends it to the server that notifies the other clients with the used one.
After the effect makes the changes, the server notifies all the clients with the changes done.



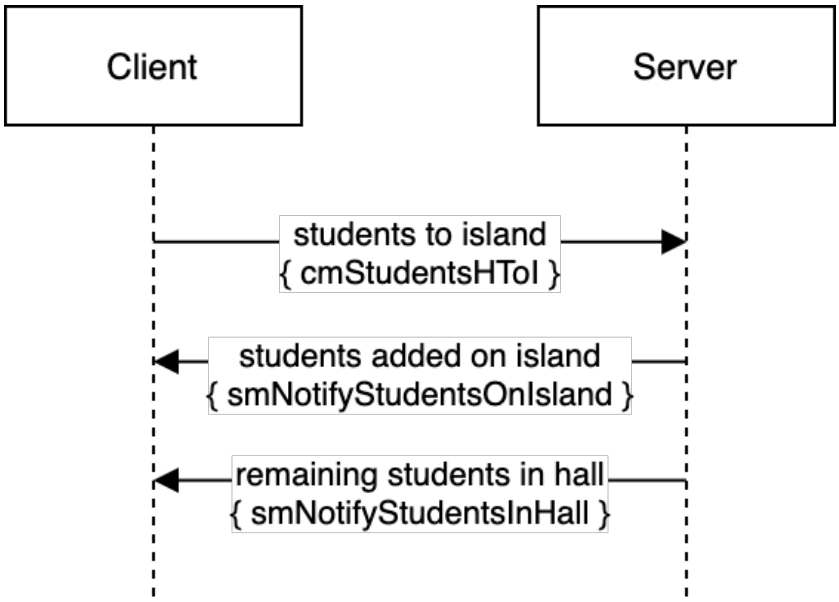
Students to dining hall

The client communicates the students that he wants to move from the hall to the dining hall.
If the mode selected is the expert one, the server also notifies with the coins taken.



Students to island

If the moved students are less than three, the client can choose to move other students from the hall to the island. Then the server notifies the other clients of the remaining students in the hall.

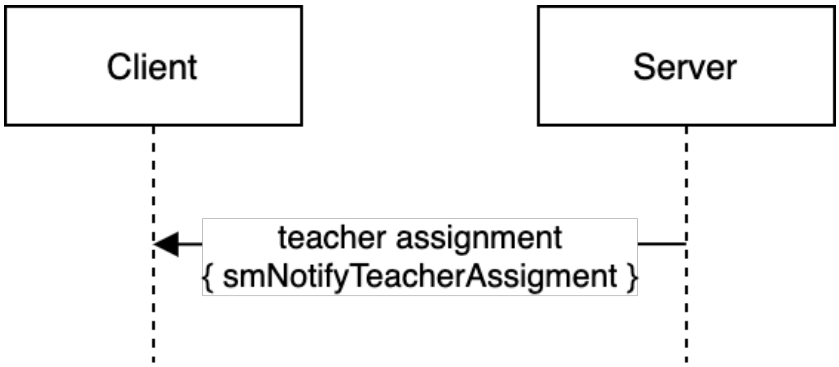


Activate character card

(Only in expert mode)
The current player also in this moment can activate a character card.

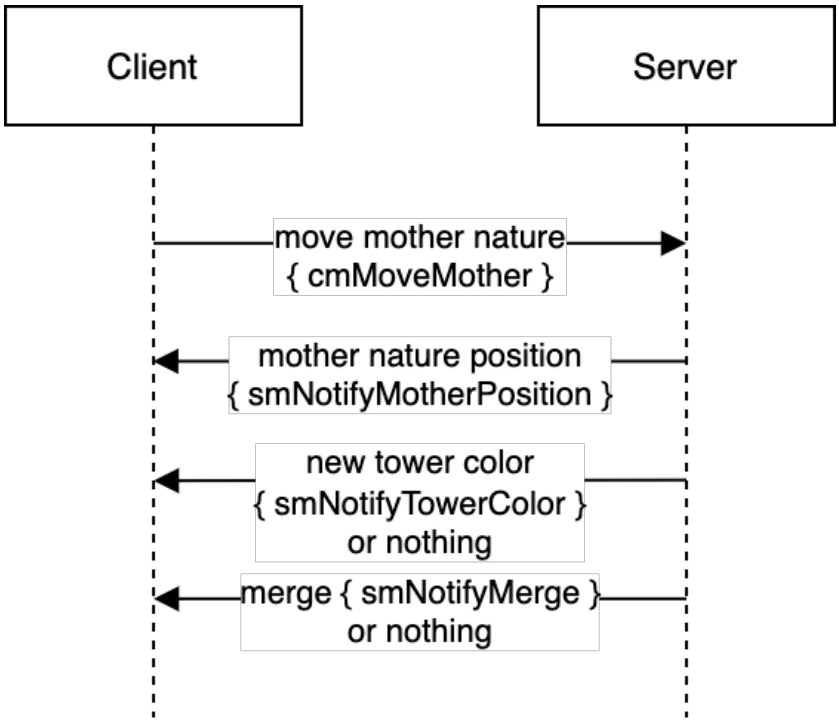
Teacher assignment

If the current client has more students of a color in the dining hall than the other clients, it will be notified from the server that a teacher is assigned to a client.



Move mother nature

The client sends the position where it wants to put mother nature.
Then notify each client of the new position of mother nature.
If the tower color is changed, a notify with the new color is sent, or a notify of no changed color.
If the merge is done, a notify with the unified islands is sent, otherwise a notify of no merge.

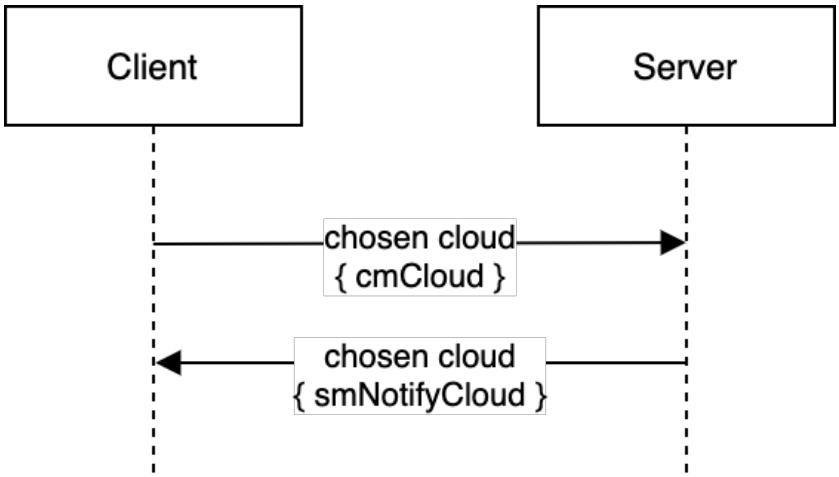


Activate character card

(Only in expert mode)
The current player also in this moment can activate a character card.

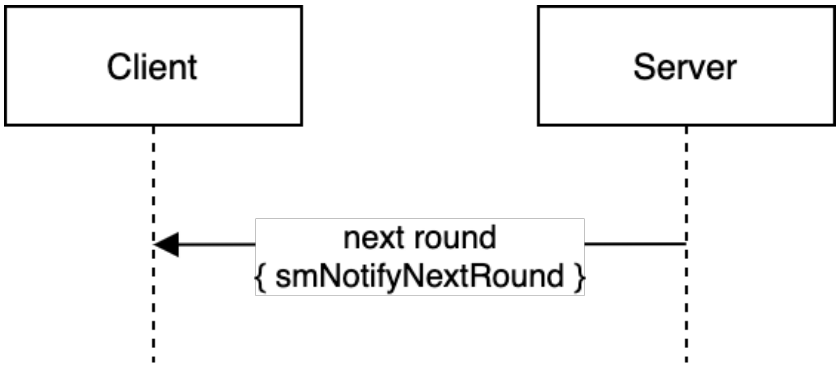
Choose cloud

If there's more than a cloud with students on it, the client chooses one to take all the students from.
A message with the cloud taken, and with also the nickname of who did it, is sent to the other players. Successively the client that is the next one in the player order knows that it's his turn.



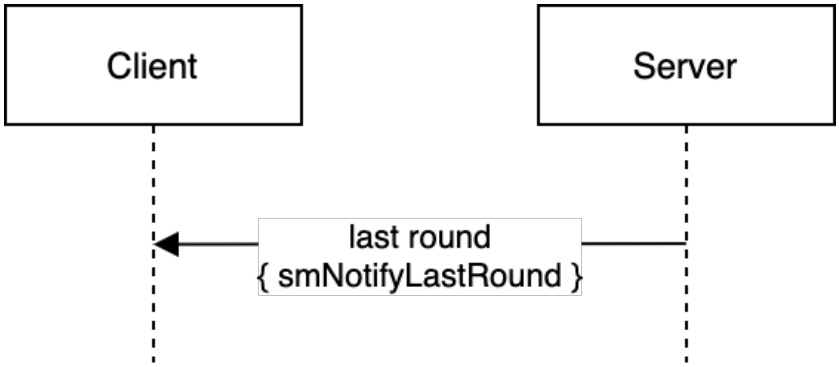
Next round

When all players have finished their turn, the server notifies them with a message saying wether the game is over or if another round is going to start.



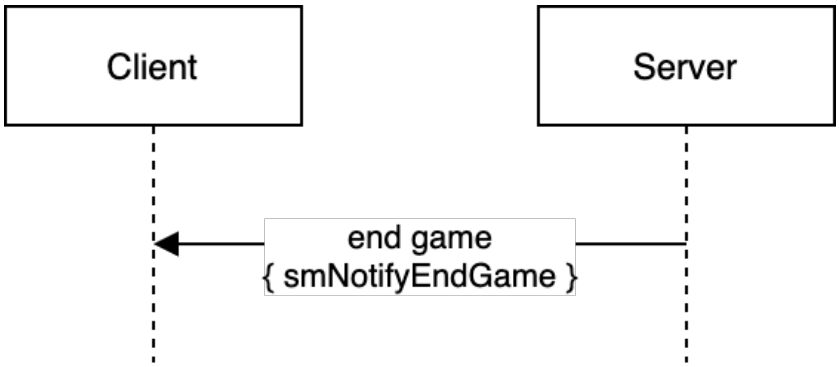
Last round

When the last player is taken from the bag, the server sends a message to communicate that it is the last round.



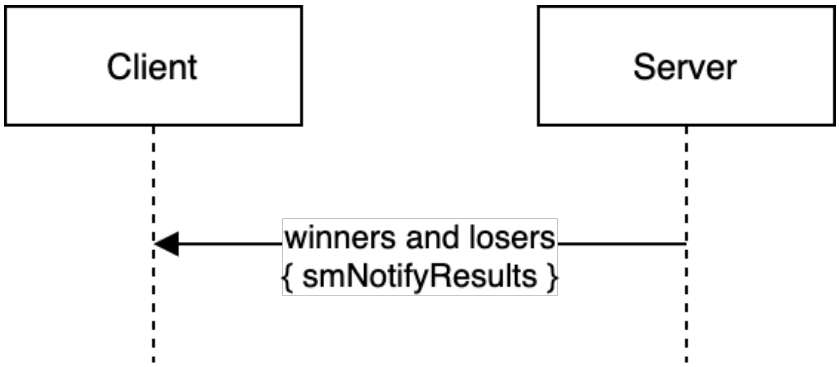
End game

When a condition of end of game is reached, the server notifies which is the condition and ends the game immediately.



Game results

After the end of the game, the server notifies the clients with a message that communicates who has won and who has lost.



List of messages:

Server messages:

// 2 constructors one for new game or game already started, the other one useful when we are waiting for players

smGameStatus { String status, ArrayList<String> players, int numOfPlayers, String mode }

smNotifyUsedDeck { String deck }

smNotifyChosenTower { int playerNick, String color }

smNotifyRefilledCloud { int cloud, ArrayList<StudentColor> students }

smNotifyUsedSupportCard { String playerNick, int cardId }

smNotifyPlayerOrder { String nickname }

smNotifyMotherPosition { int island }

smNotifyCoins { String player, int coins }

smNotifyUsedCharacterCard { String player, int cardId, int price }

smNotifyBlockOnIsland { int island, boolean blocked }

smNotifyIgnoreColor { StudentColor color, boolean ignored }

smNotifyStudentsOnCard { int cardId, ArrayList<StudentColor> students }

smNotifyBlocksOnCard { int cardId, int blocks }

smNotifyIgnoreTower { int island, boolean ignored }

smNotifyRemovedColor { StudentColor color }

smNotifyStudentsInD { String player, ArrayList<StudentColor> diningHall }

smNotifyStudentsInH { String player, ArrayList<StudentColor> hall }

smNotifyStudentsOnCloud { int cloud, ArrayList<StudentColor> students }

smNotifyStudentsOnIsland { int island, ArrayList<StudentColor> students }

smNotifyTeacherAssigment { String player, StudentColor teacher }

smNotifyTowerColor { int island, String color }

smNotifyMerge { int island1, int island2 }

smNotifySameTower { }

smNotifyNoMerge { }

smNotifyCloud { String playerNick, int cloud }

smNotifyNextRound { Boolean nextRound }

smNotifyEndGame { String cause }

smNotifyLastRound { String cause }

smNotifyResults { ArrayList<String> winners, ArrayList<String> losers }

Client messages:

cmLoginAttempt { }

cmNickname { String Nickname }

cmSetGameStatus { String mode, int numOfPlayers }

cmDeck { String deck }

cmTower { String tower }

cmSupportCard{ int id }

cmStudentsHToD { ArrayList<StudentColor> students }

cmStudentsHToI { ArrayList<StudentColor> students }

cmCCG1{ int island, StudentColor color }

cmCCG2{ int cardId }

cmCCG3{ int cardId, int movements }

cmCCG4{ int cardId, StudentColor color }

cmCCG5{ int cardId, int island }

cmCCG6{ int cardId, ArrayList<StudentColor> exchangeStudents1, ArrayList<StudentColor> exchangeStudents2 }

cmMoveMother { int position }

cmCloud{ int cloud }